

Universidad La Salle - Arequipa
Facultad de Ingeniería
Escuela Profesional de Ingeniería de Software

MTD

Memoria Técnica Descriptiva

EnglishFlow

Aplicación Interactiva para el Aprendizaje de Inglés

Proyecto Final

Curso: Desarrollo de Videojuegos

Semestre: VIII

Docente: Marco Antonio Camacho Alatrasta

Estudiante: Josué Carlos Alberto Ramos Suyoc

Arequipa, Perú
15 de julio de 2026

Índice

1. Objetivo	2
2. Título del Software	2
3. Tecnologías Utilizadas	2
4. Requisitos Funcionales	2
4.1. Estado de Implementación	3
5. Diagramas del Sistema	3
5.1. Diagrama de Componentes	3
5.2. Diagrama de Clases	4
5.2.1. Módulo de Contenido y Progreso	4
5.2.2. Módulo de Interfaz y Navegación	5
5.3. Diagramas de Flujo	5
5.3.1. Flujo: Ronda de Vocabulario	5
5.3.2. Flujo: Ronda de Gramática	6
5.3.3. Flujo: Actualización de Racha	7
5.4. Diagramas de Secuencia	8
5.4.1. Secuencia: Emparejamiento de Vocabulario	8
5.4.2. Secuencia: Cambio de Escena con Carga y Transición	9
6. Implementación	10
6.1. Estructura del Proyecto	10
6.2. Fragmentos de Código Relevantes	10
6.2.1. Servicio Central de Progreso	10
6.2.2. Armado de una Ronda de Vocabulario	11
6.2.3. Actualización de la Racha de Días	11
7. Interfaz de Usuario	12
7.1. Página Principal	12
7.2. Sección de Configuración	12
7.3. Sección de Vocabulario	13
7.4. Sección de Gramática	13
7.5. Pantalla de Carga	13

1. Objetivo

Desarrollar una aplicación interactiva de escritorio que ayude a estudiantes a practicar inglés mediante mini-juegos organizados por nivel (B1 y B2 del Marco Común Europeo), registrando el progreso de cada usuario y motivando la práctica constante a través de un sistema de rachas y recomendaciones.

2. Título del Software

EnglishFlow – Aplicación Interactiva para el Aprendizaje de Inglés.

Repositorio del proyecto: <https://github.com/JosueRamosS/EnglishFlow>.

3. Tecnologías Utilizadas

El proyecto EnglishFlow ha sido desarrollado utilizando las siguientes tecnologías:

- **Motor de desarrollo:** Unity 6 (6000.4.5f1).
- **Lenguaje:** C#.
- **Interfaz de usuario:** TextMeshPro (TMP) y Unity UI (uGUI).
- **Entrada:** Input System (soporte de teclado y mouse).
- **Animaciones y transiciones:** DOTween (DG.Tweening).
- **Persistencia de datos:** PlayerPrefs (almacenamiento local).
- **Contenido de vocabulario:** *The Oxford 3000 by CEFR level* (Oxford University Press).
- **Contenido de gramática:** *British Council – EAQUALS Core Inventory for General English*.

4. Requisitos Funcionales

El sistema EnglishFlow contempla los siguientes requisitos funcionales:

- **RF-01:** Menú principal con navegación a los módulos, operable con teclado y mouse.
- **RF-02:** Configuración de nivel de inglés (B1/B2), idioma de las instrucciones (ES/EN) y volumen.
- **RF-03:** Pantalla de carga con carga asíncrona de escenas y barra de progreso.
- **RF-04:** Módulo de Vocabulario: emparejar palabras en inglés con su definición.
- **RF-05:** Módulo de Gramática: completar oraciones eligiendo la opción correcta entre tres.
- **RF-06:** Módulo de Lectura. (*En desarrollo.*)
- **RF-07:** Módulo de Escritura. (*En desarrollo.*)
- **RF-08:** Registro del progreso por módulo (mejor puntaje, número de intentos y porcentaje de dominio).
- **RF-09:** Transiciones visuales (*fade in/out*) entre pantallas.
- **RF-10:** Pantalla de Resumen con racha de días de práctica y recomendación del módulo a reforzar.

4.1. Estado de Implementación

La siguiente tabla resume el estado real de cada requisito a la fecha de esta entrega:

RF	Descripción breve	Estado
RF-01	Menú y navegación	Completo
RF-02	Configuración	Completo
RF-03	Carga asíncrona	Completo
RF-04	Vocabulario	Completo
RF-05	Gramática	Completo
RF-09	Transiciones <i>fade</i>	Completo
RF-08	Registro de progreso	Operativo; falta la vista consolidada
RF-10	Resumen y racha	Lógica en código; UI de la escena pendiente
RF-06	Lectura	Fuera de alcance (priorización)
RF-07	Escritura	Fuera de alcance (priorización)

Nota

El motor de progreso (RF-08) está operativo: el puntaje y el dominio se guardan y se reflejan al terminar cada ronda de Vocabulario y Gramática. La pantalla de Resumen (RF-10) tiene su lógica implementada en `ModuloResumen.cs` y compila, pero su interfaz aún no está conectada en la escena. Lectura y Escritura (RF-06 y RF-07) se dejaron fuera del alcance de esta entrega para priorizar la profundidad de los módulos jugables.

5. Diagramas del Sistema

Los siguientes diagramas representan la arquitectura, la estructura y el funcionamiento del sistema EnglishFlow.

5.1. Diagrama de Componentes

El sistema se organiza en capas. La **Capa de Presentación** está formada por las escenas de Unity (Carga, Menú Principal, Configuración, Vocabulario, Gramática y Resumen) y sus elementos de interfaz contruidos con TextMeshPro y Unity UI.

La **Capa de Controladores** contiene los scripts `MonoBehaviour` que gobiernan cada pantalla: `MenuPrincipal`, `Configuracion`, `ModuloVocabulario`, `ModuloGramatica` y `ModuloResumen`.

La **Capa de Servicios** agrupa la lógica transversal: `GameProgress` (progreso y configuración sobre `PlayerPrefs`), `CargadorDeEscenas` (punto único de cambio de escena), `PantallaDeCarga` (carga asíncrona) y `ScreenFader` (transiciones con `DOTween`).

La **Capa de Datos** está compuesta por los bancos de contenido `BancoVocabulario` y `BancoGramatica`, separados por nivel B1/B2.

Finalmente, las **Librerías Externas** (Unity Engine, TextMeshPro, Input System y DOTween) proporcionan el motor, la interfaz, la entrada y las animaciones.

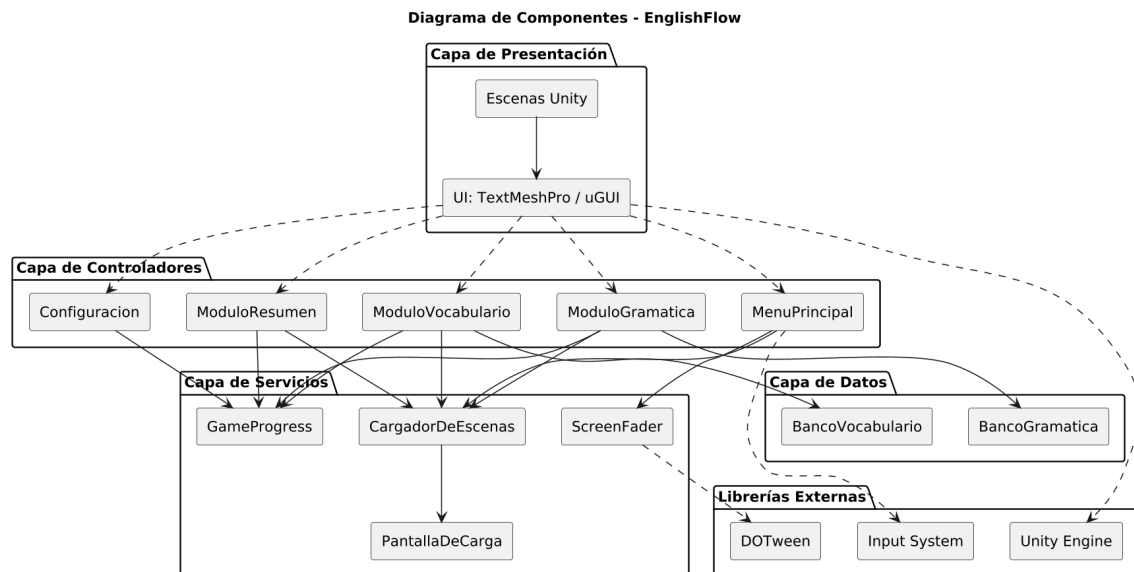


Figura 1: Diagrama de Componentes del Sistema EnglishFlow
Fuente: *Elaboración propia*

5.2. Diagrama de Clases

El diagrama de clases presenta la estructura orientada a objetos del sistema, mostrando las clases principales y sus relaciones.

5.2.1. Módulo de Contenido y Progreso

Este módulo contiene la clase estática **GameProgress**, que centraliza el guardado y la lectura de la configuración y del progreso. Los bancos **BancoVocabulario** y **BancoGramatica** exponen los datos (**ParVocabulario** y **PreguntaGramatica**) según el nivel activo. Los controladores **ModuloVocabulario**, **ModuloGramatica** y **ModuloResumen** consumen estos bancos y actualizan el progreso mediante **GameProgress**.

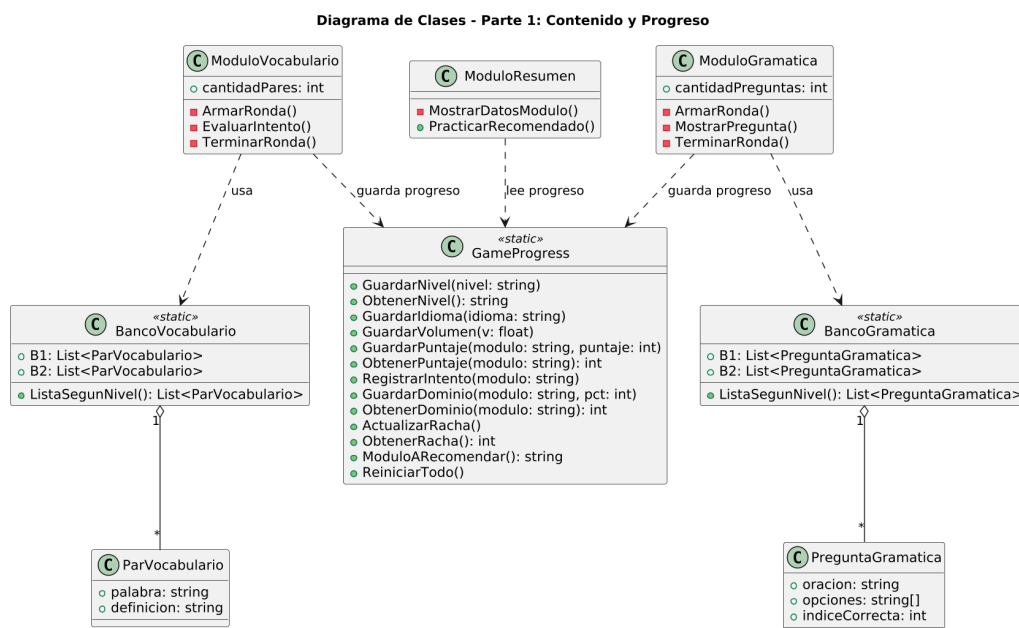


Figura 2: Diagrama de Clases – Parte 1: Contenido y Progreso
Fuente: *Elaboración propia*

5.2.2. Módulo de Interfaz y Navegación

Este módulo agrupa las clases responsables de la navegación y del sistema visual. **CargadorDeEscenas** y **PantallaDeCarga** gestionan el cambio de escena; **ScreenFader** realiza las transiciones *fade*. Del sistema visual neobrutalista se encargan **BotonNeobrutalista** y **PaletaColores**, mientras que **TarjetaVocabulario** y **OpcionGramatica** representan los elementos seleccionables de cada módulo.

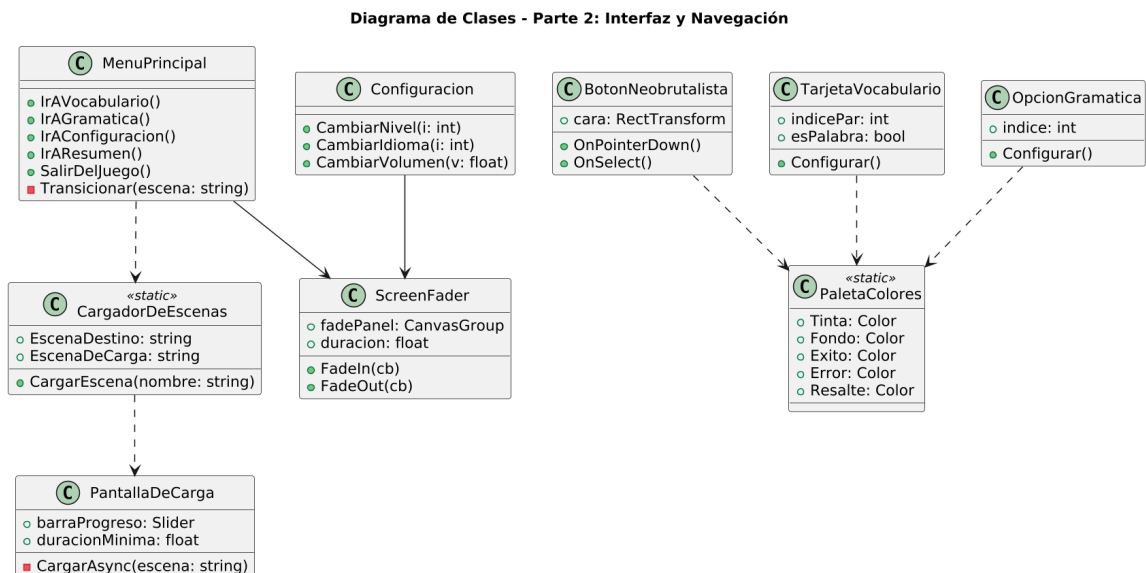


Figura 3: Diagrama de Clases – Parte 2: Interfaz y Navegación

Fuente: Elaboración propia

5.3. Diagramas de Flujo

Los diagramas de flujo describen, paso a paso, el proceso de las funcionalidades principales.

5.3.1. Flujo: Ronda de Vocabulario

El proceso inicia al entrar al módulo. El sistema toma el banco del nivel activo, lo mezcla y selecciona la cantidad de pares configurada. Reparte las palabras y las definiciones en dos columnas con órdenes independientes. El usuario selecciona una palabra y una definición; si coinciden, el par se marca como resuelto (verde); si no, se marca error (rojo) y puede reintentar. Al resolver todos los pares, el sistema calcula aciertos, errores y dominio, guarda el progreso y muestra el panel de resumen.

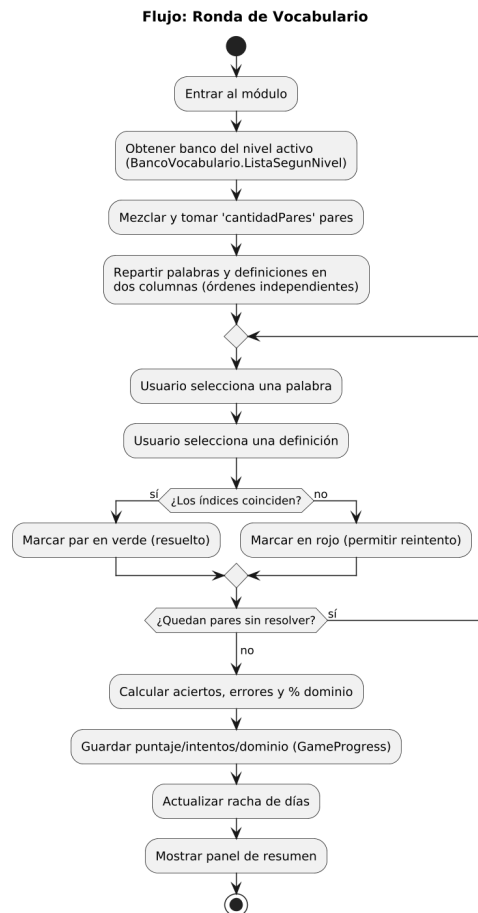


Figura 4: Diagrama de Flujo: Ronda de Vocabulario

Fuente: Elaboración propia

5.3.2. Flujo: Ronda de Gramática

De estructura similar al anterior, el sistema toma preguntas del banco del nivel activo, las mezcla y muestra una por una. Para cada pregunta, el usuario elige una de las tres opciones; el sistema evalúa el acierto, aplica el color de *feedback* y avanza a la siguiente. Al terminar, calcula el resultado, guarda el progreso y muestra el resumen.

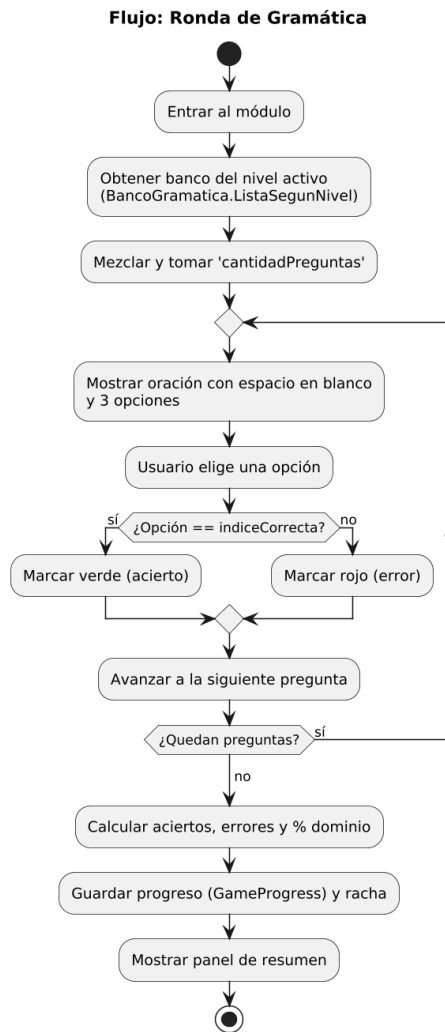


Figura 5: Diagrama de Flujo: Ronda de Gramática

Fuente: Elaboración propia

5.3.3. Flujo: Actualización de Racha

Al completar una ronda, el sistema registra la práctica del día. Si ya se contó hoy, no hace nada. Si la última práctica fue el día anterior, incrementa la racha en 1; si pasó más de un día, la reinicia a 1. Finalmente guarda la fecha y el valor de la racha.

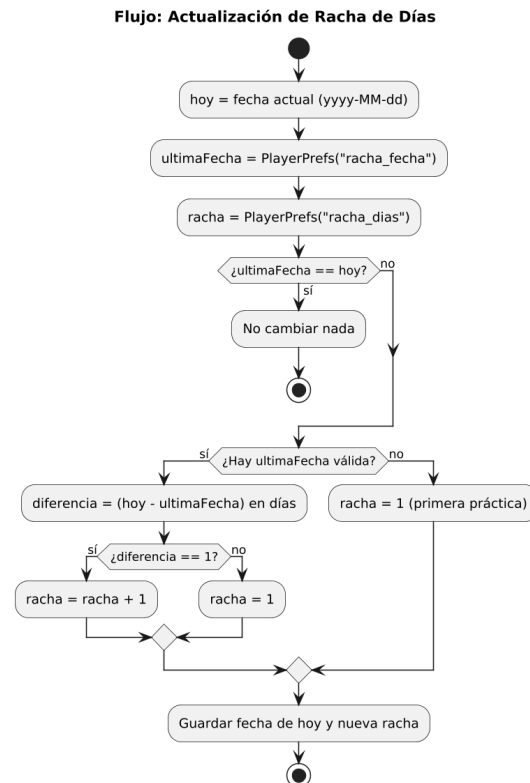


Figura 6: Diagrama de Flujo: Actualización de Racha

Fuente: Elaboración propia

5.4. Diagramas de Secuencia

Los diagramas de secuencia muestran la interacción temporal entre los componentes.

5.4.1. Secuencia: Emparejamiento de Vocabulario

Intervienen el Usuario, `ModuloVocabulario`, `TarjetaVocabulario` y `GameProgress`. El usuario selecciona una palabra y una definición; `ModuloVocabulario` compara los índices, aplica el color de resultado sobre las tarjetas y, al completarse la ronda, invoca a `GameProgress` para guardar puntaje, intentos y dominio.

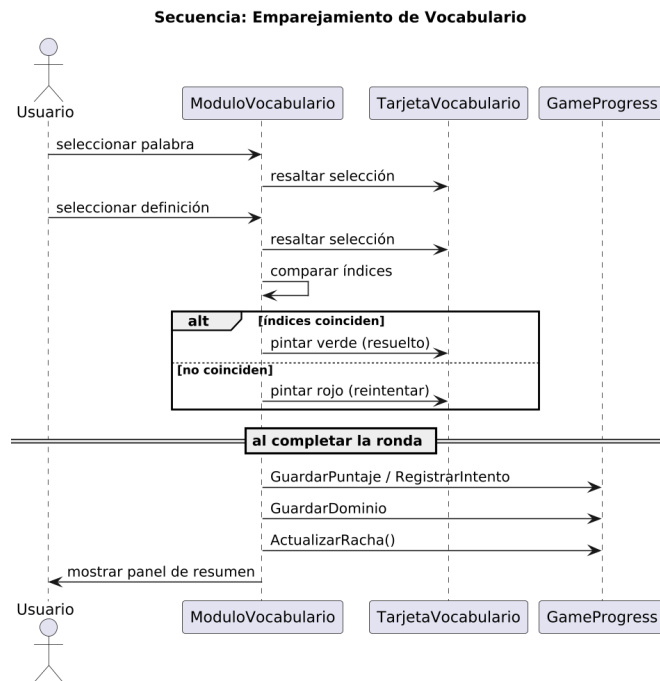


Figura 7: Diagrama de Secuencia: Emparejamiento de Vocabulario
Fuente: Elaboración propia

5.4.2. Secuencia: Cambio de Escena con Carga y Transición

Intervienen el Usuario, el controlador de la pantalla actual, **ScreenFader**, **CargadorDeEscenas** y **PantallaDeCarga**. Al pulsar un botón de navegación, el controlador solicita a **ScreenFader** un *fade out*; al terminar, llama a **CargadorDeEscenas**, que guarda el destino y abre la escena de Carga. **PantallaDeCarga** realiza la carga asíncrona con barra de progreso y activa la escena destino, donde **ScreenFader** hace el *fade in*.

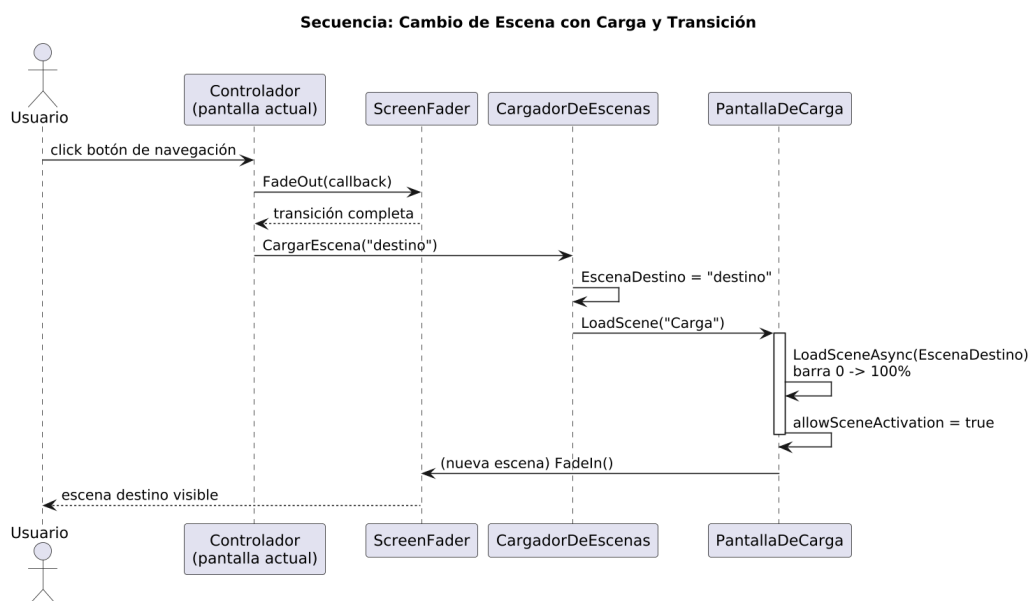


Figura 8: Diagrama de Secuencia: Cambio de Escena
Fuente: Elaboración propia

6. Implementación

6.1. Estructura del Proyecto

El proyecto está organizado según la estructura estándar de Unity, separando las escenas, los scripts y los recursos:

```
EnglishFlow/
Assets/
  Scenes/
    Carga.unity
    MenuPrincipal.unity
    Configuracion.unity
    ModuloVocabulario.unity
    ModuloGramatica.unity
    Resumen.unity
  Scripts/
    GameProgress.cs           # Servicio central de progreso y config (
    PlayerPrefs)
    CargadorDeEscenas.cs     # Punto unico de cambio de escena
    PantallaDeCarga.cs       # Carga asincrona con barra de progreso
    ScreenFader.cs           # Transiciones fade in / fade out (DOTween)
    MenuPrincipal.cs         # Menu principal y navegacion
    Configuracion.cs         # Pantalla de configuracion
    ModuloVocabulario.cs     # Logica del modulo de Vocabulario
    TarjetaVocabulario.cs    # Tarjeta seleccionable (palabra / definicion)
    BancoVocabulario.cs      # Banco de pares palabra-definicion (B1/B2)
    ModuloGramatica.cs       # Logica del modulo de Gramatica
    OpcionGramatica.cs       # Boton de opcion de respuesta
    BancoGramatica.cs        # Banco de preguntas de opcion multiple (B1/B2)
    ModuloResumen.cs         # Pantalla de resumen, racha y recomendacion
    BotonNeobrutalista.cs    # Efecto visual del boton
    PaletaColores.cs         # Paleta de colores del sistema visual
    DeseleccionarConMouse.cs # Utilidad de seleccion de UI
  Prefabs/ Resources/ Fonts/ ...
Packages/                   # manifest.json (dependencias)
ProjectSettings/           # Configuracion del proyecto Unity
```

6.2. Fragmentos de Código Relevantes

6.2.1. Servicio Central de Progreso

La clase estática `GameProgress` guarda el progreso en `PlayerPrefs`, conservando siempre el mejor valor de puntaje y de dominio por módulo:

```
1 public static class GameProgress
2 {
3     // Guarda el puntaje solo si es mejor que el anterior.
4     public static void GuardarPuntaje(string modulo, int puntaje)
5     {
6         string clave = "punt_" + modulo;
7         int actual = PlayerPrefs.GetInt(clave, 0);
8         if (puntaje > actual)
9             PlayerPrefs.SetInt(clave, puntaje);
10        PlayerPrefs.Save();
11    }
12
13    // Porcentaje de dominio (0-100). Guarda el mejor.
14    public static void GuardarDominio(string modulo, int porcentaje)
15    {
16        porcentaje = Mathf.Clamp(porcentaje, 0, 100);
17        string clave = "dom_" + modulo;
```

```
18     int actual = PlayerPrefs.GetInt(clave, 0);
19     if (porcentaje > actual)
20         PlayerPrefs.SetInt(clave, porcentaje);
21     PlayerPrefs.Save();
22 }
23 }
```

6.2.2. Armado de una Ronda de Vocabulario

El controlador toma el banco del nivel activo, lo mezcla y reparte las palabras y definiciones en dos columnas con órdenes independientes, de modo que no queden alineadas en la misma fila:

```
1 void ArmarRonda()
2 {
3     List<ParVocabulario> banco =
4         new List<ParVocabulario>(BancoVocabulario.ListaSegunNivel());
5     Mezclar(banco);
6
7     int cantidad = Mathf.Min(cantidadPares, banco.Count);
8     paresActivos = banco.GetRange(0, cantidad);
9
10    // Dos ordenes independientes para las dos columnas.
11    List<int> ordenPalabras = OrdenMezclado(cantidad);
12    List<int> ordenDefiniciones = OrdenMezclado(cantidad);
13
14    foreach (int i in ordenPalabras)
15        CrearTarjeta(contenedorPalabras, paresActivos[i].palabra, i, true);
16    foreach (int i in ordenDefiniciones)
17        CrearTarjeta(contenedorDefiniciones, paresActivos[i].definicion, i,
18            false);
19
20    ActualizarProgreso();
21 }
```

6.2.3. Actualización de la Racha de Días

La racha se incrementa solo en días consecutivos y se reinicia si el usuario deja de practicar más de un día:

```
1 public static void ActualizarRacha()
2 {
3     string hoy = DateTime.Now.ToString("yyyy-MM-dd");
4     string ultimaFecha = PlayerPrefs.GetString("racha_fecha", "");
5     int racha = PlayerPrefs.GetInt("racha_dias", 0);
6
7     if (ultimaFecha == hoy) return; // ya se conto hoy
8
9     if (!string.IsNullOrEmpty(ultimaFecha) &&
10        DateTime.TryParse(ultimaFecha, out DateTime fechaAnterior))
11     {
12         int diferencia = (DateTime.Now.Date - fechaAnterior.Date).Days;
13         racha = (diferencia == 1) ? racha + 1 : 1;
14     }
15     else racha = 1; // primera vez que practica
16
17     PlayerPrefs.SetString("racha_fecha", hoy);
18     PlayerPrefs.SetInt("racha_dias", racha);
19     PlayerPrefs.Save();
20 }
```

7. Interfaz de Usuario

La aplicación cuenta con una interfaz de estilo **neobrutalista** (colores planos, bordes marcados y botones con sombra) coherente en todas las pantallas gracias a la paleta centralizada en `PaletaColores`.

7.1. Página Principal

Presenta el título del proyecto y los accesos a cada módulo mediante un menú persistente. Permite navegar con mouse o con teclado, resaltando la opción activa.

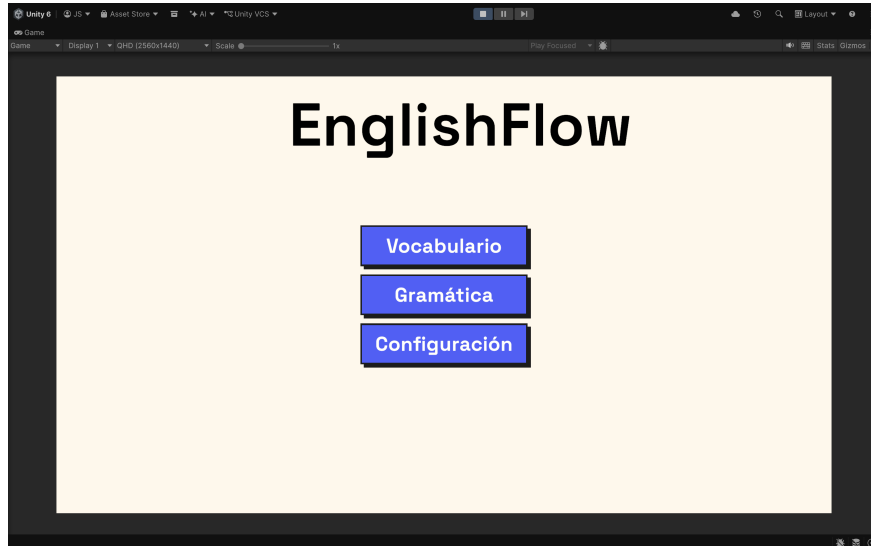


Figura 9: Menú Principal de EnglishFlow
Fuente: Elaboración propia

7.2. Sección de Configuración

Permite elegir el nivel de inglés (B1/B2), el idioma de las instrucciones y el volumen. Los cambios se guardan y aplican de inmediato mediante `GameProgress`.



Figura 10: Escena de Configuración de EnglishFlow
Fuente: Elaboración propia

7.3. Sección de Vocabulario

Muestra dos columnas (palabras y definiciones) y un contador de progreso de la ronda. El resultado de cada intento se comunica con color: verde para acierto y rojo para error.

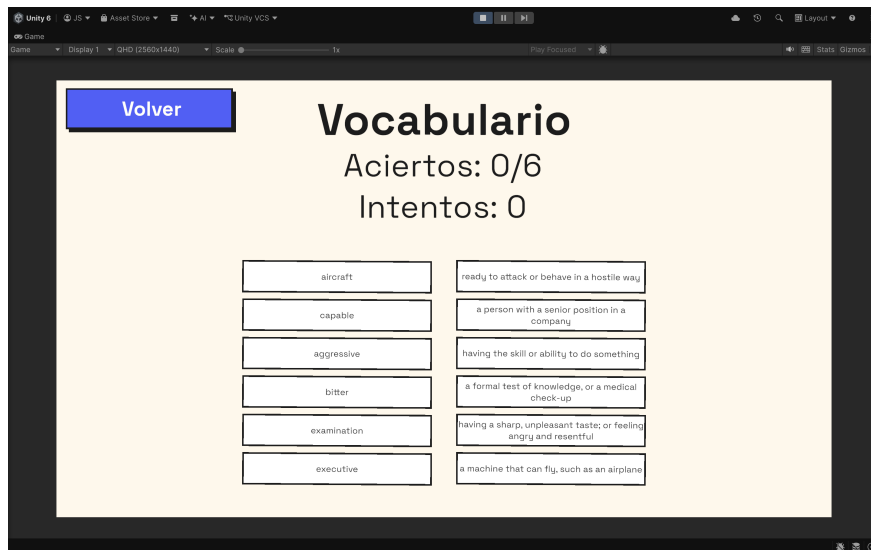


Figura 11: Escena de Vocabulario EnglishFlow
Fuente: Elaboración propia

7.4. Sección de Gramática

Presenta una oración con un espacio en blanco y tres opciones de respuesta. Tras cada selección, indica el acierto o error y avanza a la siguiente pregunta.

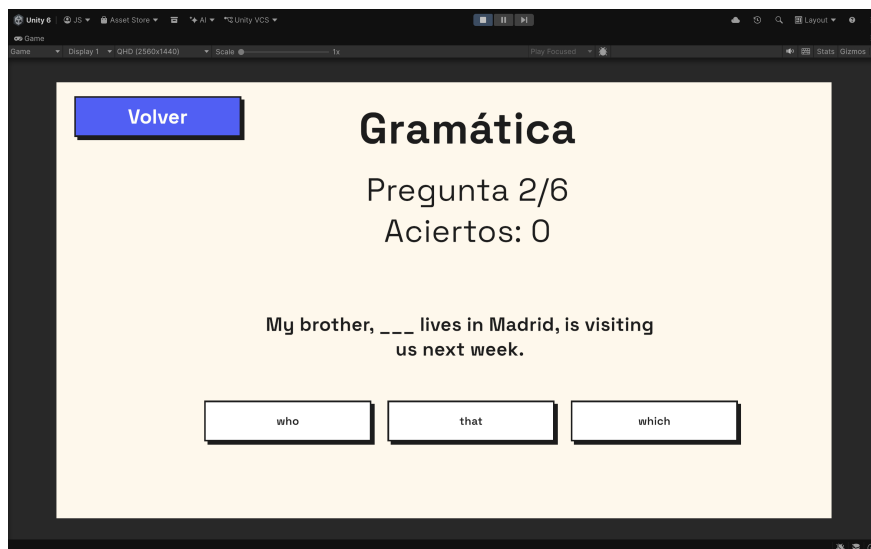


Figura 12: Escena de gramática de EnglishFlow
Fuente: Elaboración propia

7.5. Pantalla de Carga

Muestra la transición entre las escenas creadas

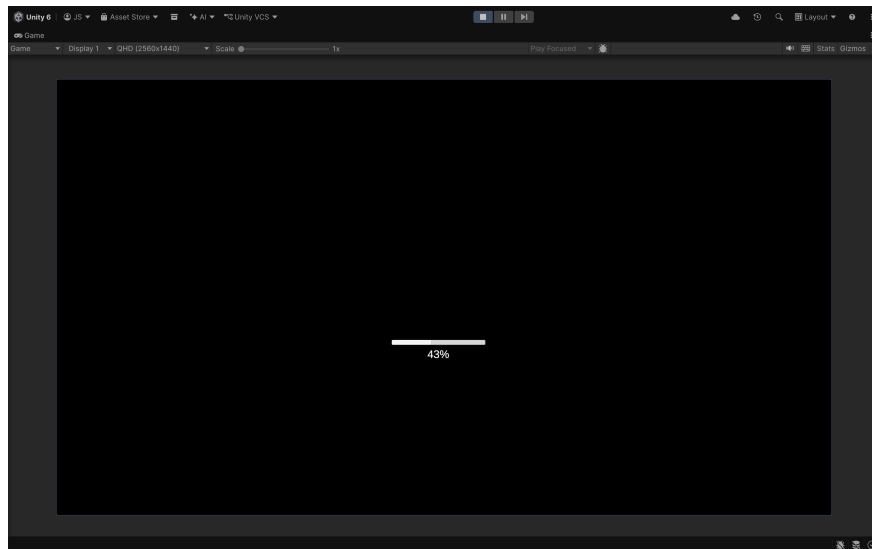


Figura 13: Pantalla de carga de EnglishFlow
Fuente: Elaboración propia